# Praxis der Softwareentwicklung

## Visualisierung von Verifikationsalgorithmen für Neuronale Netze

Praxis der Softwareentwicklung (PSE)
Wintersemester 2025/26

# Pflichtenheft

Karlsruher Institut für Technologie (KIT)
KASTEL-Institut für Informationssicherheit und Verlässlichkeit
Anwendungsorientierte Formale Verifikation
Prof. Dr. Bernhard Beckert
Am Fasanengarten 5
Gebäude 50.34
76131 Karlsruhe

**Betreuer: Philipp Kern**

**Projektteilnehmer:**

| Name | E-Mail-Adresse |
|---|---|
| Alexander Mikhaylov | uiqbn@student.kit.edu |
| Cedric Linde | unogx@student.kit.edu |
| Elias Dörr | uuxmf@student.kit.edu |
| Enrique Lopez | ukgyh@student.kit.edu |
| Paul Schepperle | ucnlo@student.kit.edu |


Karlsruhe, 18. März 2026

# Inhaltsverzeichnis

**1 Einleitung** **5**

**2 Zielbestimmung** **6**
2.1 Musskriterien . . . . . 6
2.2 Sollkriterien . . . . . 6
2.3 Kannkriterien . . . . . 7
2.4 Abgrenzungskriterien . . . . . 7

**3 Produkteinsatz** **8**

**4 Produkt- und Entwicklungsumgebung** **9**

**5 Funktionale Anforderungen** **10**
5.1 Grundlegendes . . . . . 10
5.2 Ein- und Ausgabe . . . . . 11
5.3 Diagrammansicht . . . . . 11
5.4 Netzansicht . . . . . 12

**6 Produktdaten** **13**

**7 Produktleistungen** **14**
7.1 Zeit . . . . . 14
7.2 Genauigkeit . . . . . 15

**8 Sonstige nichtfunktionale Anforderungen** **16**
8.1 Sicherheitsanforderungen . . . . . 16
8.2 Datenschutzanforderungen . . . . . 16
8.3 Urheber- und Markenrechte . . . . . 17

**9 Qualitätsanforderungen** **18**
9.1 Funktionalität . . . . . 18
9.2 Zuverlässigkeit . . . . . 18
9.3 Benutzbarkeit . . . . . 19
9.4 Effizienz . . . . . 20
9.5 Änderbarkeit / Erweiterbarkeit . . . . . 20

9.6 Portierbarkeit . . . . . 20

**10 Globale Testfälle und Testszenarien** **22**

10.1 Funktionale Testfälle . . . . . 22

10.2 Nicht-funktionale Testfälle . . . . . 25

10.3 Testszenarien . . . . . 27

**11 Benutzeroberfläche** **29**

**12 Zeit- und Ressourcenplanung** **33**

**Glossary** **34**

# Abbildungsverzeichnis

5.1 Use-Case-Diagramm . . . . . 10

11.1 Ansichten-Diagramm . . . . . 29
11.2 Tab-Ansicht (Neuronale Netze) . . . . . 30
11.3 Konfig-Ansicht . . . . . 30
11.4 Tab-Ansicht (Diagramme) . . . . . 31
11.5 Vergleichs-Tab-Ansicht . . . . . 32
11.6 Neuronen Auswahl . . . . . 32

12.1 Gantt-Diagramm . . . . . 33

# 1 Einleitung

Während KI in immer mehr sicherheitskritischen Funktionen eingesetzt wird, steigt auch die Sorge vor möglichen Fehlfunktionen und dementsprechend auch das Bedürfnis nach Möglichkeiten, die Ausgaben von KIs zu verifizieren.

Doch die Entwicklung von dafür geeigneten Algorithmen ist oftmals schwer und aufwendig, nicht zuletzt weil sie sich bislang nicht gut testen und miteinander vergleichen lassen.
Die von uns entwickelte App hat es als Ziel, genau dieses Problem zu lösen indem sie das Verständnis von neuen Verifikationsalgorithmen erleichtert, bei deren Entwicklung hilft und die dafür notwendige Wissensvermittlung in der Lehre unterstützt.

Das Produkt soll eine Desktop Anwendung werden, mit der sich neuronale Netze und deren erreichbare Mengen visualisieren, vergleichen und verifizieren lassen können. Dabei sollen die Verifikationsalgorithmen, Eingabebounds, Netze und individuelle Knoten dynamisch austauschbar sein.

# 2 Zielbestimmung

## 2.1 Musskriterien

Das Programm muss...

⟨KM1⟩ verschiedene Netze und zugehörige Informationen einlesen können.

⟨KM2⟩ aus 2 Ansichten bestehen. Es gibt eine Ansicht, die die Netze darstellt, und eine Ansicht, die die Diagramme darstellt.

⟨KM3⟩ die Möglichkeit, über ein Interface neue Algorithmen in das Programm zu implementieren, bieten.

⟨KM4⟩ eine Auswahl von 2 Neuronen aus einem Netz unterstützen.

⟨KM5⟩ einen dynamischen Wechsel von Netzen, Verifikationsalgorithmen, den zu untersuchenden Neuronen und dem Eingabeset unterstützen.

⟨KM6⟩ die Darstellung von erreichbaren Mengen von Eingabemengen an spezifischen Neuronen in einem 2D Graphen unterstützen.

⟨KM7⟩ die Graphen als .png exportieren können.

⟨KM8⟩ eine automatische Erkennung von stark aktivierten Neuronen bei Durchläufen von Eingaben unterstützen.

⟨KM9⟩ Aktionen und Netze aus letzten Sessions speichern.

## 2.2 Sollkriterien

Das Programm soll...

⟨KS1⟩ auch eine Auswahl von 3 Neuronen mit 3D Visualisierung unterstützen.

⟨KS2⟩ die Graphen als .svg-Dateien exportieren können.

## 2.3 Kannkriterien

Das Programm kann...

⟨KK1⟩ die Möglichkeit, mehr als 3 Neuronen auszuwählen und die Abhängigkeiten darzustellen.

⟨KK2⟩ die Möglichkeit, das User Interface selbst schnell anpassen zu können, bieten.

⟨KK3⟩ den Loss bei Eingaben anzeigen.

⟨KK4⟩ die Möglichkeit, die Gewichte manuell zu ändern, unterstützen.

⟨KK5⟩ die Gewichte von Kanten visuell darstellen.

## 2.4 Abgrenzungskriterien

Das Programm soll nicht...

⟨KA1⟩ das Training von Netzen selbst durchführen.

⟨KA2⟩ die Verifikationsalgorithmen auf Korrektheit überprüfen.

⟨KA3⟩ Architekturen von Netzen, die keine neuronalen Netze mit Schichtarchitektur sind, unterstützen.

⟨KA4⟩ mehrere Sprachen bzw. das Wechseln zwischen Sprachen unterstützen.

# 3 Produkteinsatz

Folgende Annahmen werden über den Einsatz des Produkts getroffen:

- Das Produkt wird am KIT in der Forschung und Lehre von formalen Verifikationsalgorithmen verwendet, insbesondere im Bereich Künstlicher Intelligenz.
- Es darf ein grundlegendes Vorwissen über die behandelten Themen vorausgesetzt werden.
- Das Programm muss sowohl von Lehrpersonal als auch von Studierenden verwendet werden können.
- Das Programm wird auch von Personen verwendet, die nur eine kurze Einführung erhalten haben.

# 4 Produkt- und Entwicklungsumgebung

Das Produkt soll auf den folgenden Betriebssystemen laufen:

- Windows 11
- macOS 15
- Linux Ubuntu 24.04
- Linux Fedora 43

Das Programm ist eine Desktop-Anwendung. Somit sind die wichtigen Performancegrößen abhängig von dem verwendeten Gerät. Die Software muss auf einer breiten Auswahl von Laptops und Arbeitsgeräten funktionieren, wie sie typischerweise von Studierenden verwendet werden. Folgende Mindestanforderungen für Geräte müssen ausreichend sein:

- Arbeitsspeicher: 8GB
- Prozessor: Prozessor: Intel Core i5-8259U oder vergleichbare
- Festplatte: SSD

Das Projekt wird mit der PySide6-Bibliothek (Ver. 6.10) als Binding für die C++ Bibliothek Qt6 entwickelt. Es wird als Programmiersprache Python (Ver. 3.12) verwendet, für Diagramme wird die Bibliothek Matplotlib verwendet. Als Eingabeformate sollen ONNX-Dateien verwendet werden, um rohe Netzgewichte zu importieren, und vnnlib-Dateien, um Spezifikationen zu den Netzen laden zu können. Zusätzlich soll es die Möglichkeit geben, Bounds an Eingaben über csv-Dateien zu importieren.

# 5 Funktionale Anforderungen

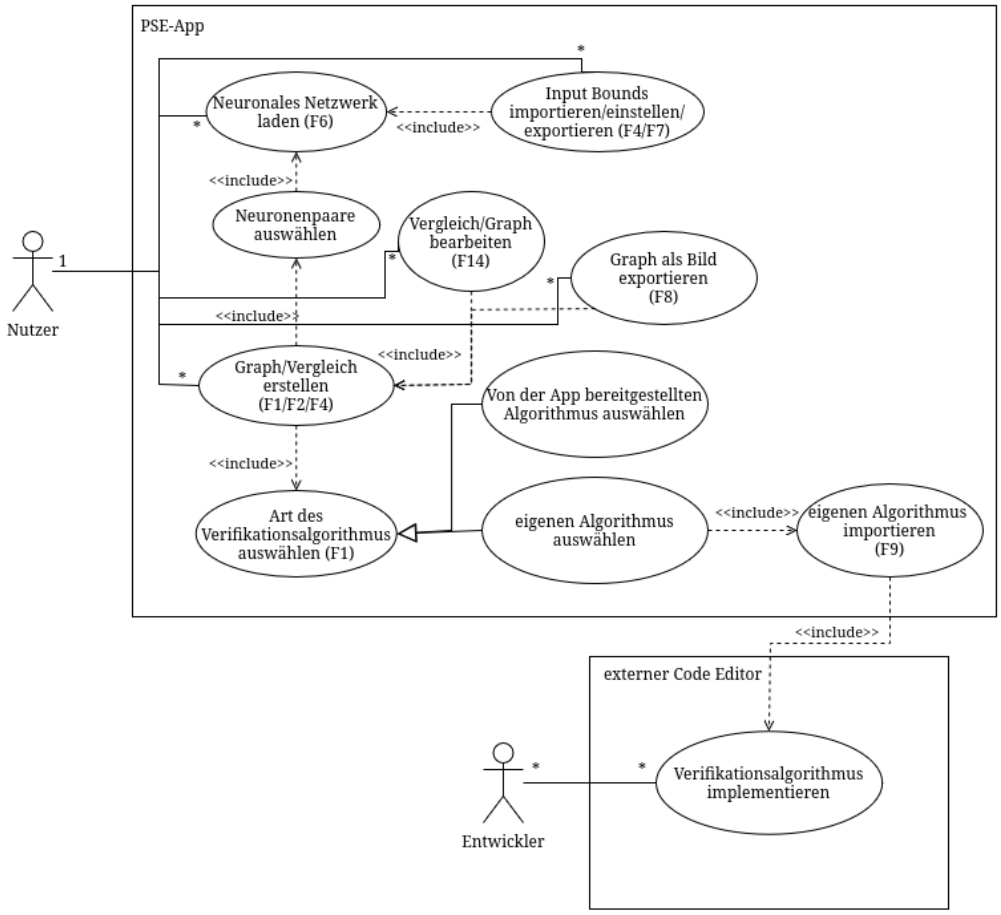


Abbildung 5.1: Use-Case-Diagramm

## 5.1 Grundlegendes

⟨F1⟩ Aus einer Auswahl von Neuronen (standardmäßig zwei) und einem ausgewählten Algorithmus soll die Software ein Diagramm erstellen können.

⟨F2⟩ Bei der Auswahl von mehreren Neuronenpaaren soll ein Diagramm pro Paar generiert und in einer Vergleichsansicht angezeigt werden.

⟨F3⟩ In der Vergleichsansicht soll es die Möglichkeit geben, Überlagerungen von Diagrammen zu erstellen.

⟨F4⟩ Der Nutzer soll in der Netzwerkansicht die Input Bounds des Netzwerkes einstellen können.

⟨F5⟩ Die Neuronen eines Paares müssen demselben Netz angehören, es sollen jedoch auch Paare aus verschiedenen Netzen miteinander vergleichbar sein.

## 5.2 Ein- und Ausgabe

⟨F6⟩ Der Nutzer soll neuronale Netze aus Dateien laden können (Format: .onnx)

⟨F7⟩ Die Input Bounds eines Netzwerkes sollen als .csv-Datei importierbar und exportierbar sein. Per .vnnlib-Datei können sowohl Input als auch Output Bounds geladen werden.

⟨F8⟩ Diagramme sollten als Bild (.png) exportierbar sein.

⟨F9⟩ Es soll eine Python-Schnittstelle zur Einbindung selbst implementierter Algorithmen existieren.

⟨F10⟩ Der Nutzer soll nach dem erneuten Starten der Software die Option haben, die zuletzt geöffnete Konfiguration (Netze, Bounds, Paare, Algorithmen) zu laden.

## 5.3 Diagrammansicht

⟨F11⟩ Die Visualisierung sollte im dreidimensionalen aus verschiedenen Richtungen möglich sein. Zwischen Vorder- und Seitenansicht kann mit einem Knopfdruck gewechselt werden.

⟨F12⟩ Ab vier Dimensionen kann der Plot in zwei- oder dreidimensionale Diagramme projeziert werden. Die zu projezierenden Dimensionen können vom Nutzer selbst gewählt werden.

⟨F13⟩ Die Diagramme können durch Draggen mit der Maus verschoben und im Dreidimensionalen gedreht werden (Azimut und Höhenwinkel).

⟨F14⟩ Die ausgewählten Neuronenpaare und Algorithmen in einem Vergleich sollten auch nach dessen Erstellung noch editierbar sein.

⟨F15⟩ Alte Vergleiche bleiben in Tabs geöffnet und können erneut angezeigt werden.

## 5.4 Netzansicht

⟨F16⟩ Zwischen den geladenen Netzen kann mithilfe von Tabs gewechselt werden.

⟨F17⟩ Der Nutzer sollte durch die 2D-Layer-Ansicht des Netzwerks mit der Maus navigieren können (Pan, Zoom).

⟨F18⟩ Interessante Neuronen sollten hervorgehoben werden, um die Auswahl zu erleichtern (z.B. durch Testdaten besonders oft aktiviert, besonders viele Verbindungen, etc.).

⟨F19⟩ Um große Netzwerke zu verwalten, soll der Nutzer nach einzelnen Neuronen suchen können (Layer, Position im Layer)

# 6 Produktdaten

Folgende Dateien werden bei Gebrauch ausschließlich lokal auf dem Rechner gespeichert:

- Projektdateien zu den zuletzt geöffneten Projekten. Darin befinden sich:
  - Die (ursprünglichen) Pfade der enthaltenen neuronalen Netze und Bounds
  - Informationen über die geöffneten Ansichten (z.B. welche Neuronenpaare oder Algorithmen)
- Error-Log-Dateien für die vergangenen Sizungen
- Die vom Nutzer ausgewählten Einstellungen in einer Konfigurationsdatei

# 7 Produktleistungen

Als generelle Leistungsverbesserung wird für alle relevanten Berechnungen die Python-Bibliothek Numpy verwendet. Diese ist im Vergleich zu den Standard-Python-Operatoren sehr viel schneller und bietet die Möglichkeit höherer Präzision bei Berechnungen.

## 7.1 Zeit

Ein Großteil der verbrauchten Rechenzeit des Programms stammt voraussichtlich aus...

- dem Laden und die Visualisierung des neuronalen Netzes.
- dem Ausführen der Algorithmen auf dem Netz.
- dem Visualisieren der Algorithmus-Ergebnisse.

Dabei hängt die Laufzeit insbesondere von der Größe des verwendeten neuronalen Netzes ab.

Dementsprechend lassen sich für die zeitliche Leistung folgende Ansprüche formulieren:

- Die zeitlich limitierenden Funktionen müssen so effizient wie möglich umgesetzt werden
- Alle weiteren Funktionen des Programms müssen so implementiert werden, dass ihr zeitlicher Mehraufwand minimiert wird, sodass der Nutzer außerhalb der begrenzenden Funktionen keine erheblichen Verzögerungen erfährt

Als Teil der Effizienz der zeitkritischen Lösungen gilt die Vermeidung von unnötiger doppelter Arbeit einzuhalten. Dafür werden Ergebnisse von deterministischen Algorithmen innerhalb einer Session temporär zwischengespeichert, um bei identischen Eingaben eine sofortige Wiederherstellung des Plots zu ermöglichen.

## 7.2 Genauigkeit

Bei deterministischen Algorithmen (die gecached werden) müssen bei mehrfachem Laden des Ergebnisses immer die gleichen Resultate herauskommen.

Die Genauigkeit jeglicher Visualisierungen ist durch die Genauigkeit des vom Nutzer implementierten Algorithmus begrenzt. Dabei muss das Programm die vom Nutzer gelieferten Daten genau genug verarbeiten und anzeigen, um nicht selbst einzuschränkend zu wirken.

Prinzipiell kann der Nutzer eine beliebig hohe Genauigkeit erreichen, indem er mehr Richtungen hinzufügt.

# 8 Sonstige nichtfunktionale Anforderungen

## 8.1 Sicherheitsanforderungen

Zur Gewährleistung der Sicherheit werden folgende zwei Hauptmaßnahmen getroffen:

- Die Software läuft komplett offline, bietet also potenziellen Angreifern keine besondere Angriffsfläche
- Zur Entwicklung des Programms werden nur verbreitete und sichere Python-Bibliotheken verwendet. Um dies zu gewährleisten, wird die snyk PyPI Package Database verwendet. Dabei gelten eine Sicherheitsbewertung von "no known security issues" und eine Instandhaltungsbewertung von "sustainable" als Mindestanforderung

Das größte Sicherheitsrisiko des Programms ist jedoch die Möglichkeit des Nutzers, eigene Python-Programme auszuführen. Die grundlegende Verantwortung dafür, nur sichere Algorithmen auszuführen, liegt nicht bei dem Programm, sondern bei dessen Nutzer. Dabei werden folgende Sicherheitsmaßnahmen vonseiten des Programms implementiert:

- Es werden keine Algorithmen ausgeführt, die der Nutzer nicht explizit dafür auswählt
- Dem Nutzer wird aktiv seine Verantwortung und das Risiko beim Ausführen von fremdem Code bewusst gemacht

## 8.2 Datenschutzanforderungen

Die durch das Programm erhobenen Produktdaten werden weder intern noch an Dritte weitergegeben. Außerhalb der Produktdaten erfolgt keine Datenerhebung.

## 8.3 Urheber- und Markenrechte

Es werden alle Urheber- und Markenrechte für verwendeten Ressourcen eingehalten. Dies gilt sowohl für das Endprodukt, als auch für die zwischen den Phasen des Projekts stattfindenden Vorträge. Es besteht der Anspruch, dass am Ende des Projekts der gesamte Code unter einer Open-Source-Lizenz veröffentlicht wird. Dabei wird die konkrete Lizenz angepasst, um die Lizenzen von verwendeten Ressourcen nicht zu verletzen (z.B. Copyleft-Open-Source-Lizenzen)

# 9 Qualitätsanforderungen

## 9.1 Funktionalität

⟨QF1⟩ Fehlerhafte Eingabedateien oder nicht unterstützte Dateitypen müssen ohne Absturz des Programms erkannt und mit einer klaren, nicht-technischen Fehlermeldung abgewiesen werden, die dem Nutzer die Art des Problems (z.B. ungültige ONNX-Datei) mitteilt.

⟨QF2⟩ Bei wiederholter Berechnung von deterministischen Algorithmen mit identischen Eingaben liefert das Programm identische, reproduzierbare Ergebnisse.

⟨QF3⟩ Nicht kritische Fehler (z.B. Ladeversuch einer ungültigen Datei) und Warnungen (z.B. Neuronenzahl-Grenzwert) müssen in einer separaten, für den Nutzer zugänglichen Log-Datei im .log Format protokolliert werden, um die Fehleranalyse zu unterstützen.

## 9.2 Zuverlässigkeit

⟨QZ1⟩ Stabilität der Anwendung unter Last: Bei Durchführung der in Abschnitt 10 definierten Standard-Testszenarien über einen Zeitraum von 1 Stunde darf das Programm nicht abstürzen.

⟨QZ2⟩ Grenzwert für Neuronenzahl: Das Programm muss ab einer Größe von 100.000 Neuronen einen Empfehlungshinweis ausgeben, der den Nutzer auf mögliche, lange Rechenzeiten aufmerksam macht und empfiehlt, nur fortzufahren, wenn genügend Rechenressourcen vorhanden sind.

⟨QZ3⟩ Der Nutzer muss die Möglichkeit haben, dem Programm ein Speicherlimit in Prozent des verfügbaren freien Arbeitsspeichers zuzuweisen, um das Auftreten eines MemoryError zu vermeiden. Wird das zugewiesene Speicherlimit überschritten, wird die Ausführung des Algorithmus automatisch abgebrochen und eine entsprechende Fehlermeldung ausgegeben.

⟨QZ4⟩ Umgang mit Abbrechen eines Algorithmus: Bei Ausführung eines Visualisierungs-Algorithmus muss dem Nutzer stetig die Möglichkeit geboten werden, diese abzubrechen. Wenn dies geschieht, muss die Berechnung automatisch abgebrochen werden, ohne dass die gesamte Anwendung abstürzt oder nicht mehr reagiert.

⟨QZ5⟩ Wiederherstellung / Datenintegrität: Tritt ein nicht behebbarer Fehler auf, der zum Beenden der Anwendung führt, dürfen die zuletzt geladenen Netzwerke und die aktuelle Konfiguration nicht korrumpiert werden.

## 9.3 Benutzbarkeit

⟨QB1⟩ System Usability Scale (SUS): Ein SUS-Scale von mindestens 70 muss in der Qualitätssicherungsphase (QS) erreicht werden.

⟨QB2⟩ Die Bedienoberfläche muss gemäß der in Abschnitt 3 definierten Zielgruppe (Forscher/Studenten mit technischer Expertise) konsistent in Fachterminologie gehalten werden.

⟨QB3⟩ Dokumentationsstandard: Die Schnittstellendefinition muss in einem strukturierten Dokument vorliegen und ein lauffähiges Code-Beispiel für das Hinzufügen eines einfachen Algorithmus enthalten.

⟨QB4⟩ Eindeutige Kennzeichnung: Bei einer Visualisierung muss der Name des zugehörigen neuronalen Netzes und des Neuronenpaars (eventuell als Abkürzung) permanent und eindeutig sichtbar sein.

⟨QB5⟩ Mehrere Diagramme können ausgewählt und automatisch auf einen gemeinsamen, geeigneten Maßstab skaliert werden. Darüber hinaus kann der Maßstab einzelner Diagramme jederzeit wieder auf den ursprünglichen Standardmaßstab zurückgesetzt werden.

⟨QB6⟩ Konsistentes Design: Die visuelle Gestaltung der Anwendung (Farben, Schriftarten, Abstände, Icons) muss über alle Ansichten (Netzansicht, Diagrammansicht, Dialoge) und Bildschirmgrößen hinweg konsistent sein und den Standards eines modernen Desktop-Tools entsprechen, um eine hohe ästhetische Wahrnehmung und geringe kognitive Belastung zu gewährleisten.

⟨QB7⟩ Feedback-Mechanismen: Das Programm muss bei allen Benutzeraktionen, die länger als 1 Sekunde dauern (z. B. Laden von Netzen, Start der Visualisierung), visuelles Feedback (z. B. Ladebalken, Fortschrittsanzeige) liefern, um dem Nutzer Transparenz über den aktuellen Systemzustand zu geben und Wartezeiten subjektiv zu verkürzen.

⟨QB8⟩ Die Funktionalität des Programm darf nicht abhängig davon sein, ob das Gerät, auf dem es läuft, mit dem Internet verbunden ist.

## 9.4 Effizienz

Folgende Limits werden für Ladezeiten gesetzt (dabei beziehen sich alle Angaben auf eine Standardlast des Systems):

⟨QE1⟩ Start des Programms: 2 Sekunden

⟨QE2⟩ Speichern eines Graphen als Bild: 0.5 Sekunden

⟨QE3⟩ Speichern des Projekts als Datei: 1 Sekunde

⟨QE4⟩ Laden und Anzeigen eines Netzes: 10 Sekunden

⟨QE5⟩ Visualisierung mit der Box-Methode auf einem Standardnetz: 1 Sekunde

⟨QE6⟩ Visualisierung mit der Zonotop-Methode auf einem Standardnetz: 3 Sekunden

## 9.5 Änderbarkeit / Erweiterbarkeit

⟨QÄ1⟩ Die klar definierte Python-Schnittstelle erlaubt das modulare Implementieren und Anpassen von Verifikationsalgorithmen

⟨QÄ2⟩ Konfiguration (Nutzer-Präferenzen) müssen in Einstellungen änderbar sein, dabei müssen triviale Änderungen umgesetzt werden, ohne die Anwendung neu zu starten.

⟨QÄ3⟩ Unit-Test-Abdeckung: Alle Klassen und Methoden der kritischen Kernlogik müssen zu mindestens 75% durch Unit-Tests abgedeckt sein. Die Einhaltung muss durch einen automatisierten Test-Report (Code Coverage Tool) verifiziert werden.

⟨QÄ4⟩ Struktur und Code-Qualität: Die Einhaltung des Code-Standards PEP 8 muss durch den Einsatz von automatisierten Linter-Tools im Entwicklungsprozess erzwungen werden.

## 9.6 Portierbarkeit

- Unterstützte Zielplattformen: Die Anwendung muss volle Funktionalität und die Erfüllung aller grundlegenden Leistungen auf folgenden spezifischen Betriebssystemversionen gewährleisten:

⟨QP1⟩ Windows 11

⟨QP2⟩ macOS 15 (auf Intel- und Apple Silicon-Architekturen)

⟨QP3⟩ Linux Ubuntu 24

⟨QP4⟩ Linux Fedora 43

- Standardisiertes App-Format: Für jede Zielplattform muss ein nativ ausführbares, standardisiertes Verteilungsformat bereitgestellt werden, das die Installation für den Endnutzer vereinfacht:

⟨QP5⟩ Windows: Installer (.exe)
⟨QP6⟩ macOS: Disk Image (.dmg)
⟨QP7⟩ Linux: Portable oder .deb-Paket

# 10 Globale Testfälle und Testszenarien

## 10.1 Funktionale Testfälle

⟨TF1⟩ ONNX-Netz laden

- *Vorbedingung*: Gültige ONNX-Datei vorhanden.
- *Ablauf*: Anwendung starten → „Netz laden“ → Datei wählen → bestätigen.
- *Ergebnis*: Netz wird geladen und in 2D angezeigt (Layer/Neuronen sind sichtbar).
- *Eigenschaften*: Funktionalität (korrektes Laden unterstützter Netze)
- *Anforderungen*: F6

⟨TF2⟩ Mehrere Netze laden und dynamisch wechseln

- *Vorbedingung*: Mindestens zwei gültige ONNX-Dateien.
- *Ablauf*: Netz A laden → Netz B laden → im UI zwischen A/B umschalten (Dropdown/Tab).
- *Ergebnis*: Beide Netze sind verfügbar, der Wechsel ist ohne Fehler möglich.
- *Eigenschaften*: Funktionalität, Benutzbarkeit (Arbeitsabläufe mit mehreren Netzen)
- *Anforderungen*: F6, F16

⟨TF3⟩ – Fehlerhafte Netz-Datei erkennen

- *Vorbedingung*: Ungültige/defekte ONNX- oder nicht unterstützte Datei.
- *Ablauf*: „Netz laden“ → fehlerhafte Datei auswählen.
- *Ergebnis*: Verständliche Fehlermeldung, kein Absturz, Netz wird nicht übernommen.
- *Eigenschaften*: Funktionalität (Fehlererkennung für Eingabedateien), Zuverlässigkeit (robust gegen fehlerhafte Eingaben)
- *Anforderungen*: F6

⟨TF4⟩ – Input Bounds laden

- *Vorbedingung*: Netz geladen, gültige Input-Bound-Datei vorhanden.
- *Ablauf*: „Input Bounds laden“ → Datei auswählen.
- *Ergebnis*: Daten werden akzeptiert und stehen für die Visualisierung bereit.
- *Eigenschaften*: Funktionalität
- *Anforderungen*: F7

⟨TF5⟩ – Input Bounds editieren

- *Vorbedingung*: Neuronales Netz geladen.
- *Ablauf*: Tab des Netzes aufrufen → Bounds editieren.
- *Ergebnis*: Bounds wurden verändert.
- *Eigenschaften*: Funktionalität
- *Anforderungen*: F4

⟨TF6⟩ – Fehlerhafte Testbounds erkennen

- *Vorbedingung*: Falsch formatierte oder leere Testbound-Datei.
- *Ablauf*: „Daten laden“ → fehlerhafte Datei wählen.
- *Ergebnis*: Fehlermeldung, keine Übernahme, Programm bleibt stabil.
- *Eigenschaften*: Funktionalität, Zuverlässigkeit (Fehlertoleranz)
- *Anforderungen*: F7

⟨TF7⟩ – Neuronenauswahl

- *Vorbedingung*: Einsetzbares Netz geladen, Visualisierungsprozess gestartet, Netz in 2D-Netzansicht sichtbar.
- *Ablauf*: Mindestens 2 Neuronen per Mausklick oder Suche (Layer/Index) auswählen.
- *Ergebnis*: Ausgewählte Neuronen sind klar hervorgehoben (z.B. durch Farbe).
- *Eigenschaften*: Funktionalität, Benutzbarkeit.
- *Anforderungen*: F1, F17, F19

⟨TF8⟩ – Visualisierung mit Algorithmus

- *Vorbedingung*: Einsetzbares Netz geladen.
- *Ablauf*: Visualisierung erstellen → Knotenpaar hinzufügen → Algorithmus auswählen → „Visualisierung starten“.
- *Ergebnis*: Korrekte Visualisierung der Bound-Menge, Form entspricht Algorithmus, keine Fehlermeldung.
- *Eigenschaften*: Funktionalität (korrekte Implementierung der Algorithmen), Effizienz
- *Anforderungen*: F1

⟨TF9⟩ – Graphen verschiedener Netze vergleichen

- *Vorbedingung*: Mehrere einsetzbare Netze geladen.
- *Ablauf*: Visualisierung erstellen → Knotenpaare hinzufügen → je Knotenpaar verschiedenes Netz auswählen → „Visualisierung starten“ → Zwei Diagramme überlagern → Knotenpaare und Algorithmus tauschen → „Visualisierung starten“ Zwei Diagramme überlagern →

- *Ergebnis*: Beide Visualisierungen werden parallel angezeigt, unabhängige Interaktion je Seite, kein Absturz, nach Tausch der Knotenpaare und des Algorithmus werden korrekte, neue Diagramme erstellt.
- *Eigenschaften*: Funktionalität, Produktivität, Benutzbarkeit
- *Anforderungen*: F2, F3, F5,F14

⟨TF10⟩ – Export als PNG

- *Vorbedingung*: Visualisierung ist sichtbar.
- *Ablauf*: „Export als PNG…“ → Speicherort auswählen → bestätigen.
- *Ergebnis*: Datei wird erzeugt, enthält den aktuellen Plot vollständig und in ausreichender Qualität und ist an der gewählten Stelle gespeichert.
- *Eigenschaften*: Funktionalität, Effizienz / Effektivität (Standardaufgabe „Analyse → Export“)
- *Anforderungen*: F8

⟨TF11⟩ – Flüssige Navigation (Pan/Zoom)

- *Vorbedingung*: Netz oder Diagramm ist geöffnet.
- *Ablauf*: Pan mit Maus-Drag, Zoom mit Mausrad/Zoom-Buttons, ggf. Ansicht zurücksetzen.
- *Ergebnis*: Navigation ist flüssig, keine Grafikfehler, UI reagiert direkt.
- *Eigenschaften*: Benutzbarkeit (intuitive, minimalistische GUI), Effizienz (interaktive Performance)
- *Anforderungen*: F13, F17

⟨TF12⟩ – Neuen Algorithmus ausführen

- *Vorbedingung*: Neue Algorithmus-Implementierung als .py-Datei verfügbar, einsetzbares Netz geladen.
- *Ablauf*: Visualisierung starten → Neuronen auswählen → neuen Algorithmus im Dropdown wählen → Visualisierung generieren.
- *Ergebnis*: Algorithmus erscheint im UI, wird korrekt ausgeführt, bestehende Funktionen bleiben unverändert.
- *Eigenschaften*: Änderbarkeit / Erweiterbarkeit, Benutzbarkeit.
- *Anforderungen*: F9

⟨TF13⟩ – Speichern von Konfiguration

  * *Vorbedingung*: Einsetzbares Netz geladen und Diagramm generiert.
  * *Ablauf*: schließen des Programms → erneut starten des Programms
  * *Ergebnis*: Die vorher geladenen Netze und Diagramme sind bei immernoch sichtbar.
  * *Eigenschaften*: Änderbarkeit / Erweiterbarkeit, Benutzbarkeit,Effizienz.

  * *Anforderungen*: F10,F15

⟨TF14⟩ – Visualisieren von Diagrammen in höheren Dimensionen
  * *Vorbedingung*: Einsetzbares Netz geladen, Visualisierungsprozess gestartet und Tripel und n-Tupel an Knoten ausgewählt.
  * *Ablauf*: Generierung von Diagrammen → drehen des 3-D-Diagramms → Auswahl der Projektionsdimensionen des n-Tupel-Diagramms
  * *Ergebnis*: Das 3-D Diagramm lässt sich von verschiedenen Persepktiven betrachten, das n-Tupel-Diagramm lässt sich in den ausgewählten Dimensionen betrachten.
  * *Eigenschaften*: Funktionalität, Benutzbarkeit.
  * *Anforderungen*: F11, F12

⟨TF15⟩ – Hervorheben von stark aktivierten Neuronen bei Dateneingabe
  * *Vorbedingung*: Einsetzbares Netz geladen.
  * *Ablauf*: Generierung von Zufälligen Testdaten → Abarbeiten des Netzes der Testdaten → Anzeigen der am stärksten aktivierten Neuronen.
  * *Ergebnis*: Es werden die Neuronen angegeben, welche am stärksten aktiviert worden sind.
  * *Eigenschaften*: Funktionalität, Benutzbarkeit.
  * *Anforderungen*: F18

## 10.2 Nicht-funktionale Testfälle

⟨TN1⟩ – Ladezeit von Netzen/Input Bounds
  * *Vorbedingung*: Einsatzbereites Netz geladen.
  * *Ablauf*: Anwendung starten → Netz laden → Input Bounds laden → Zeit messen.
  * *Ergebnis*: Ladezeiten bleiben innerhalb der festgelegten „akzeptablen“ Grenzen, Anwendung bleibt bedienbar.
  * *Eigenschaften*: Effizienz
  * *Anforderungen*: QE4

⟨TN2⟩ – Berechnungszeit der Algorithmen
  * *Vorbedingung*: Einsatzbereites Netz geladen, Visualisierungsprozess vorbereitet
  * *Ablauf*: Vorgefertigte und eigene Algorithmen jeweils starten, dabei Zeit und Verhalten beobachten.
  * *Ergebnis*: Alle Berechnungen brauchen nicht spürbar länger als eine reine Ausführung des Algorithmus, jederzeit ist eine Abbruchmöglichkeit vorhanden (Anwendung bleibt bedienbar).
  * *Eigenschaften*: Effizienz, Zuverlässigkeit
  * *Anforderungen*: QE5, QE6

⟨TN3⟩ – Ressourcenverbrauch (RAM/CPU) mit großen Netzen

- *Vorbedingung*: Monitoring-Tool vorhanden, großes einsetzbares Netz.
- *Ablauf*: Mehrere Visualisierungen erzeugen, Ressourcen beobachten.
- *Ergebnis*: RAM/CPU bleiben unter definierten Grenzwerten, kein kontinuierlicher Anstieg, wenn möglich Caching statt Mehrfachberechnung.
- *Eigenschaften*: Effizienz
- *Anforderungen*: QZ2, QZ3

⟨TN4⟩ – Stabilität bei Fehlern und schneller Interaktion

- *Vorbedingung*: Anwendung läuft.
- *Ablauf*: Mehrfach ungültige Dateien laden, lange Berechnung starten und abbrechen, schnell zwischen Views wechseln und neue Visualisierungen starten.
- *Ergebnis*: Keine Abstürze, Fehler führen zu Meldungen statt Programmende, Ergebnisse bei identischen Eingaben bleiben deterministisch (sofern deterministische Algorithmen verwendet werden).
- *Eigenschaften*: Zuverlässigkeit (Robustheit, deterministische Ergebnisse)
- *Anforderungen*: QZ4, QZ1

⟨TN5⟩ – UI-Zuverlässigkeit auf verschiedenen Bildschirmgrößen

- *Vorbedingung*: Verschiedene Auflösungen, einsetzbares Netz und Visualisierung.
- *Ablauf*: Anwendung öffnen, Fenstergrößen variieren, Netzansicht öffnen, Diagrammansicht öffnen.
- *Ergebnis*: Icons/Logos/Menüs bleiben sichtbar, kein Überlappen, Visualisierungen bleiben nutzbar, GUI intuitiv bedienbar.
- *Eigenschaften*: Benutzbarkeit, Zufriedenheit (subjektive GUI-Wahrnehmung)
- *Anforderungen*: QB6, QB1

⟨TN6⟩ – Plattformübergreifende Funktionalität

- *Vorbedingung*: Installationen auf Windows, macOS, Linux, gleiche Version.
- *Ablauf*: Auf jedem System: Start → Netz/Bounds laden → Visualisierung → Export.
- *Ergebnis*: Gleiche Funktionalität auf allen Plattformen, es treten keine Fehler und GUI-Artefakte auf.
- *Eigenschaften*: Portierbarkeit, Zuverlässigkeit
- *Anforderungen*: QP1 - QP7

⟨TN7⟩ – Sicherheit und Offline-Betrieb

- *Vorbedingung*: Internetverbindung deaktiviert.
- *Ablauf*: Anwendung starten → Netz/Bounds laden → Visualisierung → Export.
- *Ergebnis*: Volle Funktion, keine Netzwerkzugriffe, keine Datenweitergabe.
- *Eigenschaften*: Sicherheit, Datenschutz
- *Anforderungen*: QB8

## 10.3 Testszenarien

⟨TS1⟩ – Standardanalyse eines einzelnen Netzes

- Ziel: Überprüfen, dass ein typischer Nutzer ohne zusätzliche Dokumentation ein Netz laden, Neuronen auswählen, eine Visualisierung erstellen und als PNG (PDF) exportieren kann.
- Vorbedingung: Gültiges ONNX-Netz der Standardgröße (siehe Glossar) vorhanden. Gültige Testdaten vorhanden. Anwendung ist installiert.
- Ablauf:
  - Anwendung starten.
  - Über die Standardansicht → „Netz laden“ wählen → ONNX-Datei auswählen.
  - „Input Bounds laden“ wählen → Datei auswählen.
  - Aus Bounds Testdaten generieren lassen.
  - Das Netz die Testdaten abarbeiten lassen.
  - In der Netzansicht Zoom/Pan nutzen.
  - In die Diagrammansicht wechseln.
  - Visualisierung starten → Im Visualisierungs-Panel Netz, zwei Neuronen und Algorithmus „Box“ wählen.
  - Prüfen, ob die Visualisierung nachvollziehbar ist (Achsen, Legende, usw.).
  - „Export als PNG ...“ → Speicherort auswählen → bestätigen.
  - Anwendung geöffnet lassen, Visualisierung erneut mit „Zonotop“ ausführen.
  - Anwendung schließen und wieder starten.
  - Ein Tripel und ein n-Tupel an Neuronen auswählen.
  - Diagramme erzeugen lassen, Auswahl der Projektionsebenen des n-Tupels, drehen des 3-D-Diagramms.
- Erwartetes Ergebnis:
  - Netz und Bounds werden ohne Fehlermeldung geladen.
  - Stark aktivierte Neuronen werden angezeigt.
  - Die GUI ist übersichtlich, alle benötigten Funktionen sind erreichbar.
  - Neuronen sind klar markiert, die Zuordnung zwischen Netz und Neuronenpaar ist eindeutig.
  - Box-Visualisierung wird korrekt und zügig berechnet (und bietet jederzeit die Möglichkeit, abzubrechen).
  - Exportierte PNG-Datei zeigt den aktuellen Plot vollständig und in ausreichender Qualität.
  - Keine Abstürze, erneute Visualisierung mit Zonotopen funktioniert ebenso korrekt.
  - Nach dem Neustart sind alle Diagramme noch vorhanden.
  - Das 3-D-Diagramm wurde korrekt dargestellt und gedreht, das andere Diagramm stellt die korrekten Projektionsebenen da.
- Abgedeckte Testfälle: TF1 (Netz laden), TF4 (Daten laden), TF6 (Neuronenauswahl), TF7 (Visualisierung), TF9 (Export), TF10 (Navigation),TF13(Speichern von Konfiguration), TF15(Neuronenhervorhebung),TF14(Höhere Dimensionen),

TN1/TN2 (Lade- und Berechnungszeiten), TN5 (Benutzbarkeit)

⟨TS2⟩ – Vergleich zweier großer Netze unter Last

- Ziel: Prüfen, wie sich die Anwendung bei realistischem Setup mit unterschiedlichen Netzen, großen Input Bounds und rechenintensiven Algorithmen verhält.
- Vorbedingung: Zwei große ONNX-Netze (A und B) mit vielen Layern/Neuronen. Zu jedem Netz große Input Bounds. Zeitlimits und RAM/CPU-Grenzen definiert.
- Ablauf:
  - Anwendung starten.
  - Netz A laden → zugehörige Bounds laden.
  - Netz B laden → zugehörige Bounds laden.
  - Visualisierung starten und zwei Neuronenpaare hinzufügen.
  - Für beide Paare ein „ähnliches“ Neuronenpaar aus unterschiedlichen Netzen auswählen (z.B. gleicher Layer/Position).
  - Links Algorithmus „Box“, rechts „Zonotopen“ wählen → Visualisierung starten.
  - Nach erfolgreicher Berechnung: Export beider Visualisierungen als PNG.
  - Nochmals einen Algorithmus in einem View starten und anschließend abbrechen.
  - Mehrfach hintereinander versuchen, ungültige Dateien (falscher Typ) zu laden, ohne die Anwendung neu zu starten.
- Erwartetes Ergebnis:
  - Beide großen Netze und Bounds werden geladen, Ladezeiten liegen im akzeptablen Rahmen.
  - Box- und Zonotopen-Visualisierung werden korrekt berechnet, bei hoher Auslastung bleibt das GUI reaktionsfähig.
  - Speicher- und CPU-Verbrauch steigt, bleibt aber unter den definierten Grenzen.
  - Export der beiden Plots als PNG ohne Fehler gelingt.
  - Abbruch einer langen Berechnung funktioniert, UI und restliche Funktionen bleiben stabil.
  - Mehrfach fehlerhafte Daten führen zu Fehlermeldungen, aber nie zu Abstürzen.
- Abgedeckte Testfälle: TF2 (mehrere Netze), TF8 (Multi-Netz-Vergleich), TF7 (Visualisierung komplexer Algorithmen), TF9 (Export), TF10 (Navigation), TF11 (Arbeit unter Auslastung), TF3/TF5 (fehlerhafte Dateien), TN1–TN4 (Performanz, Ressourcen, Stabilität), TN6 (plattformspezifisch – auf mehreren OS wiederholt), TN7 (Sicherheit – offline getestet)

# 11 Benutzeroberfläche

Das Programm ist in zwei verschiedene Ansichten unterteilt: Die Ansicht für neuronale Netze (grün) und die Ansicht für Diagramme und Vergleiche (blau). Damit der Nutzer beide Modi leicht auseinanderhalten kann, ändert sich beim Wechsel das Farbschema des Programms zur entsprechenden Farbe. Zwischen den Ansichten kann mit einem Schalter oben rechts in der Titelleiste jederzeit mit einem Klick gewechselt werden. Die Zusammenhänge der verschiedenen Seiten und Dialoge sind im folgenden Diagramm dargestellt:

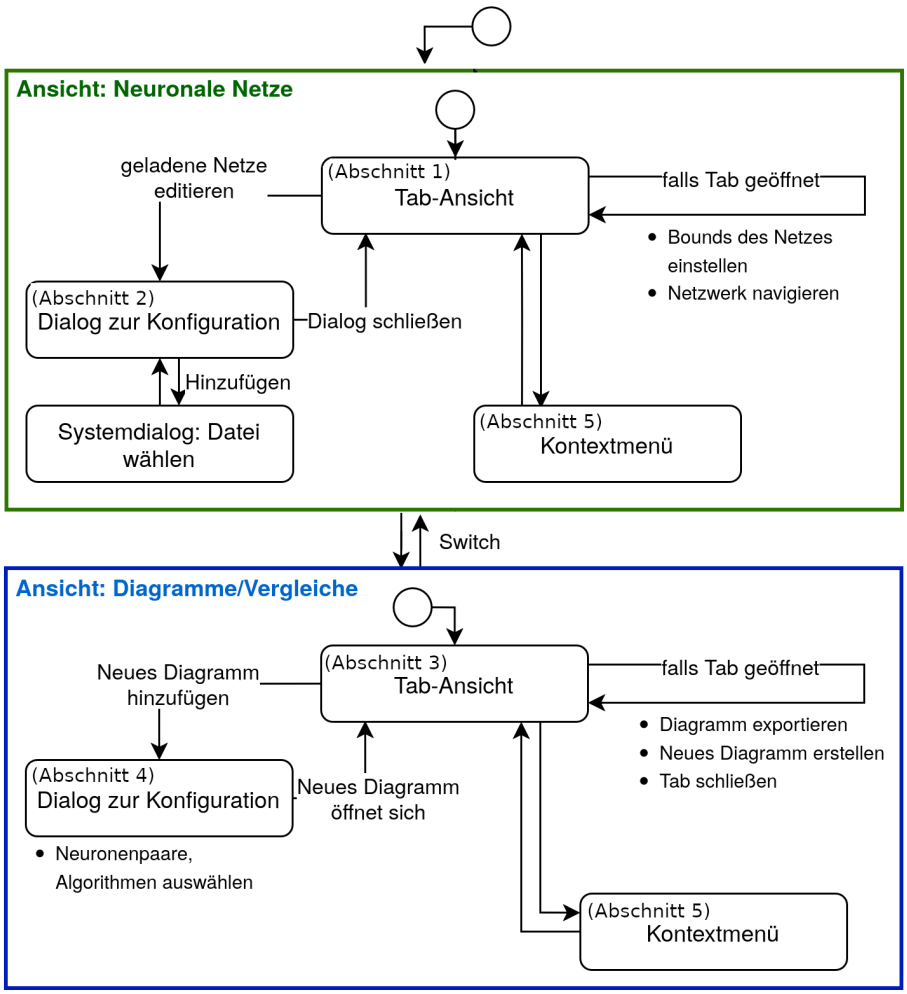


Abbildung 11.1: Ansichten-Diagramm

1. **Neuronale Netze: Tab-Ansicht**
   Die Ansicht zeigt jedes geladene neuronale Netz in einem Tab. In der oberen Leiste kann zwischen den geöffneten Netzen gewechselt werden. Das geöffnete Netz wird in einer Schichtenansicht dargestellt. Der Nutzer kann durch draggen und scrollen mit der Maus sowie mit den Zoombuttons durch das Netzwerk navigieren. Links gibt es außerdem ein ein- und ausklappbares Seitenpanel, welches Einstellungsmöglichkeiten für das geladene Netz beinhaltet. Hier können bspw. die Bounds des Netzes eingestellt werden. Durch das Edit-Icon kann der Konfigurationsdialog (2) aufgerufen werden.

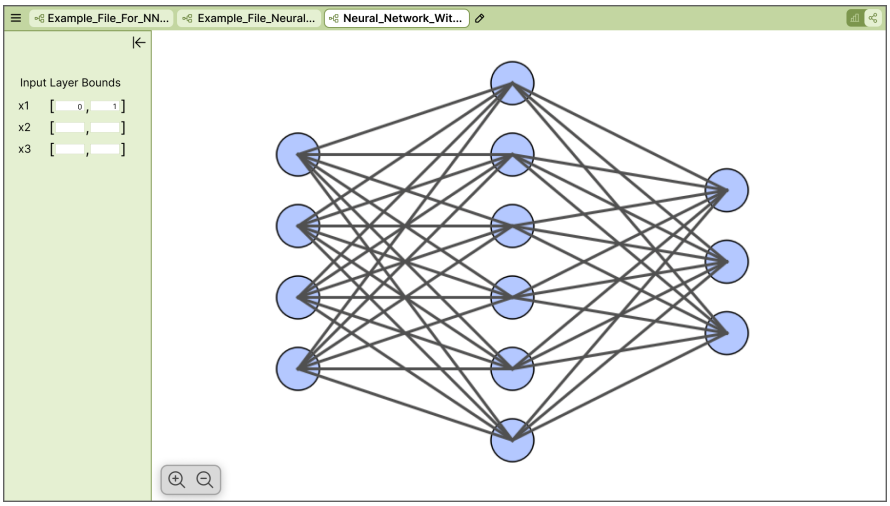


Abbildung 11.2: Tab-Ansicht (Neuronale Netze)

2. **Neuronale Netze: Konfigurationsdialog**
   Im Konfigurationsdialog können die geladenen Netzwerke verwaltet werden. Hier können neue Dateien geladen und Netze wieder entfernt werden.

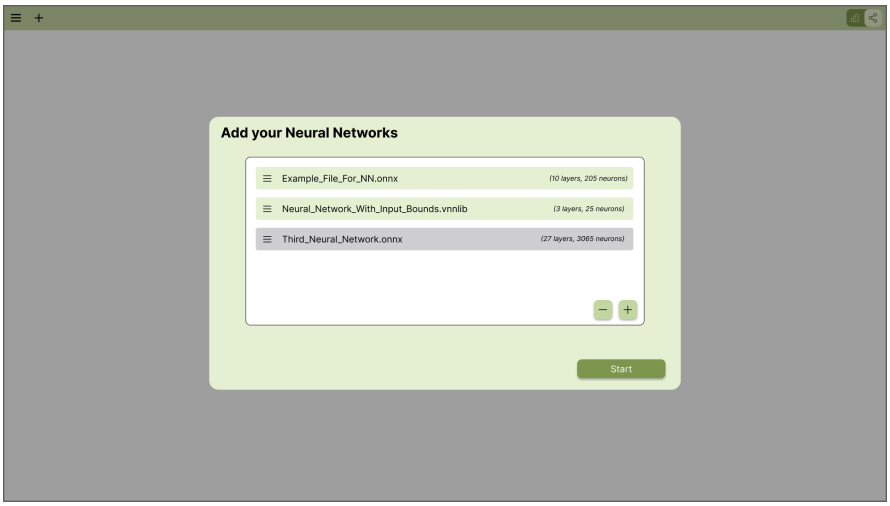


Abbildung 11.3: Konfig-Ansicht

3. **Diagramme: Tab-Ansicht**
   In dieser Ansicht lässt sich die Hauptfunktion des Programms steuern, das Erstellen von neuen Diagrammen und Vergleichen. Über das Plus-Icon kann der Dialog zum Erstellen neuer Vergleiche geöffnet werden (4). Die Ansicht eines Vergleichs ist zweigeteilt.
   Das kleinere linke Panel enthält Einstellungen für die Diagramme des Vergleichs. Hier kann ausgewählt werden, welche von den erstellten Neuronenpaaren in welchem Diagramm dargestellt werden, sowie neue Diagramme erstellt werden. Außerdem kann hier der Dialog (4) erneut aufgerufen werden, um die Neuronenpaare zu bearbeiten.
   Der rechte Teil stellt die Diagramme in einem Grid dar, indem zunächst der horizontale Platz einer Zeile gefüllt und bei Bedarf ein Umbruch in die nächste Zeile gemacht wird. Jedes Diagramm ist eingerahmt und kann über die Icons als Bild exportiert oder in einer Vollbildansicht angezeigt werden.

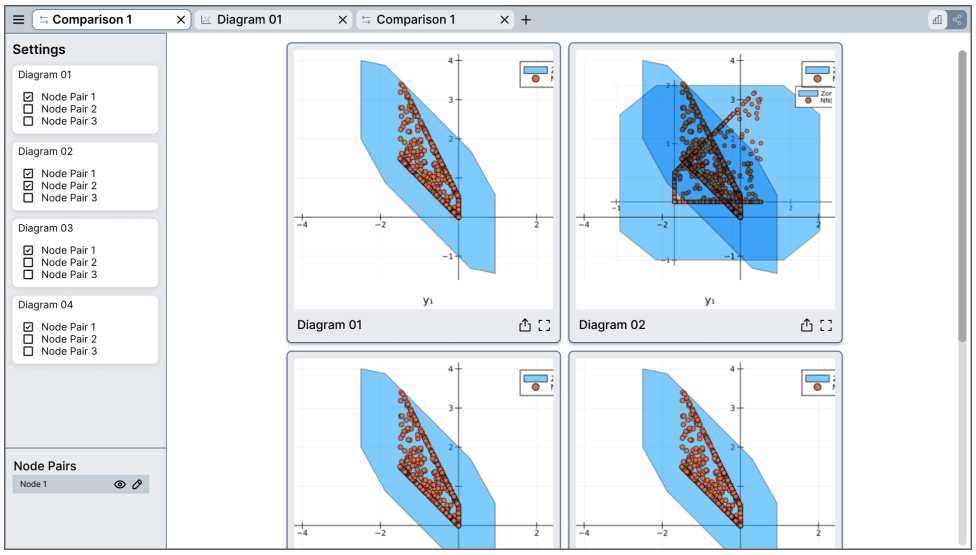


Abbildung 11.4: Tab-Ansicht (Diagramme)

4. **Diagramme: Neuen Vergleich erstellen**
   Um ein neues Diagramm oder einen neuen Vergleich zwischen mehreren Diagrammen zu erstellen, müssen Neuronenpaare mit zugeordneten Algorithmen ausgewählt werden. Der Dialog ermöglicht die schrittweise Auswahl dieser Paare. Außerdem können bereits ausgewählte Paare wieder entfernt werden. Der Button “Next” öffnet einen neuen Tab in der Ansicht (3), wenn mindestens ein Neuronenpaar ausgewählt wurde. Bei genau einem Paar ist dieser ein Diagramm-Tab, andernfalls wird ein Vergleichs-Tab erstellt.
   Beim Klick auf das Plus-Icon öffnet sich die zweite Seite des Dialogs, in der ein Neuronenpaar mit Algorithmus erstellt werden kann. Hierzu muss zunächst im oberen Dropdown das gewünschte Netz gewählt werden. Über das untere Dropdown kann der

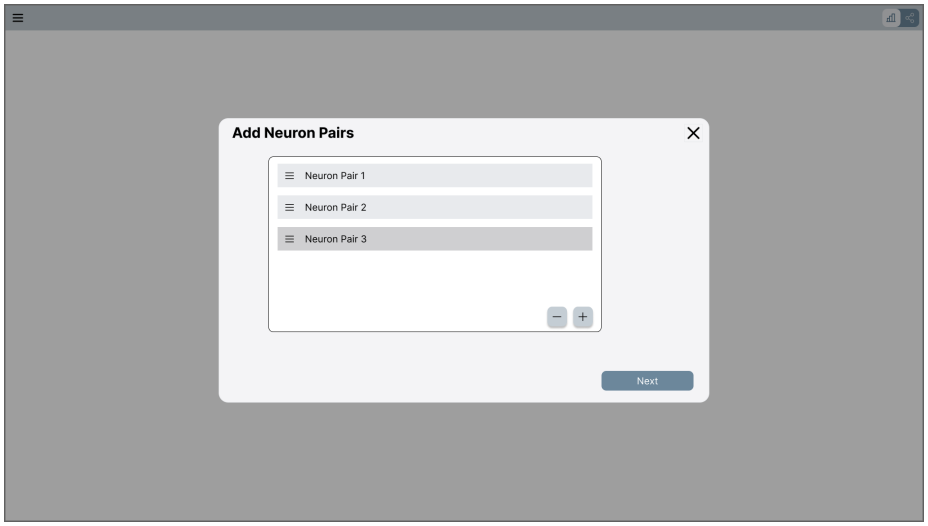


Abbildung 11.5: Vergleichs-Tab-Ansicht

gewünschte Algorithmus selektiert werden. In dem Netz können daraufhin hintereinander zwei Neuronen angeklickt werden, um diese als Paar auszuwählen. Alternativ kann der Layer- und Zeilenindex eines Neurons links in die Textfelder eingetragen werden. Um die Auswahl von Paaren zu erleichtern, sollen außerdem interessante Neuronen hervorgehoben werden.

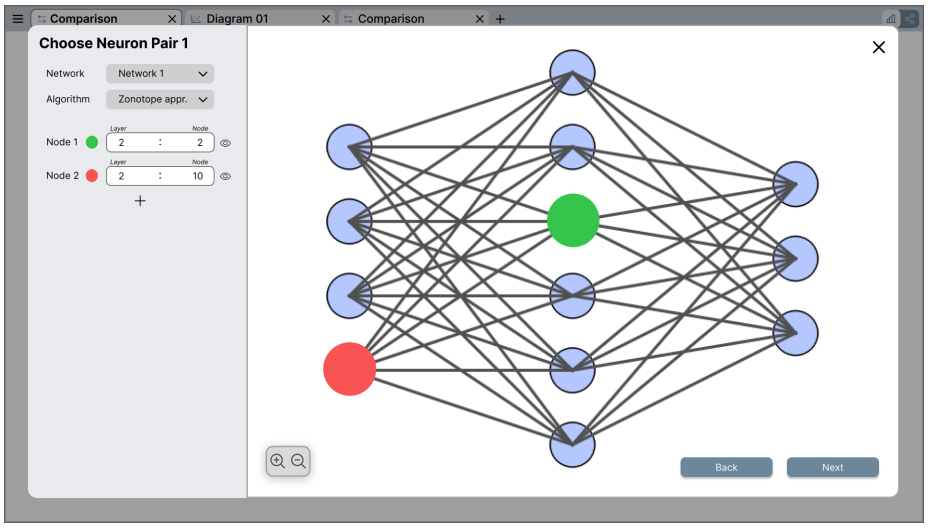


Abbildung 11.6: Neuronen Auswahl

5. **Kontextmenü**
   Durch Klicken auf die drei horizontalen Striche oben links in den Tab-Ansichten lässt sich ein Kontextmenü öffnen. Hier sind weitere globale Optionen des Programms untergebracht, wie bspw. das Öffnen eines neuen Projektes oder das Importieren eines neuen Python-Algorithmus.

# 12 Zeit- und Ressourcenplanung

| ID | Task Name | Start | End | Duration | Color |
|---|---|---|---|---|---|
| 1 | PSE | 10/11/2025 | 13/03/2026 | 124 days | |
| 2 | Pflichtenheft | 10/11/2025 | 28/11/2025 | 19 days | |
| 3 | Entwurf | 01/12/2025 | 09/01/2026 | 40 days | |
| 4 | Implementierung | 12/01/2026 | 06/02/2026 | 26 days | |
| 6 | Z1 | 12/01/2026 | 23/01/2026 | 12 days | |
| 7 | Z2 | 23/01/2026 | 30/01/2026 | 8 days | |
| 8 | Z3 | 30/01/2026 | 06/02/2026 | 8 days | |
| 9 | Z4 | 23/01/2026 | 27/01/2026 | 5 days | |
| 5 | Qualitätssicherung | 24/02/2026 | 13/03/2026 | 18 days | |

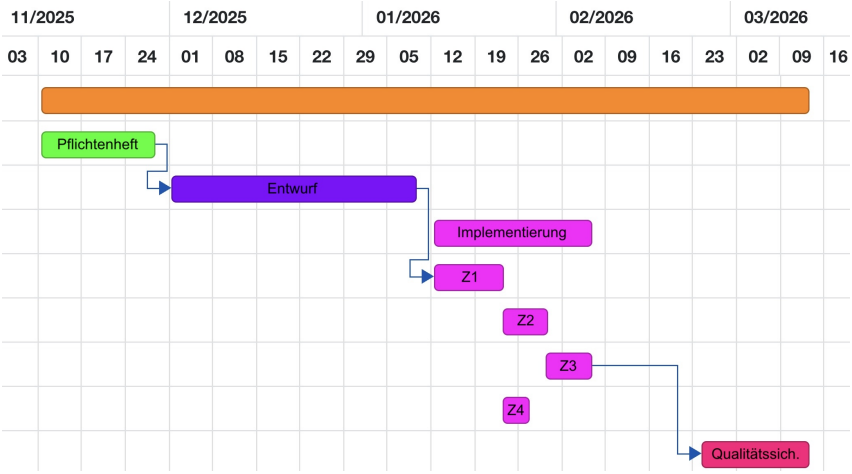


Abbildung 12.1: Gantt-Diagramm

**Zeitphasen bei Implementierung:**

- Z1: Netz Import und Darstellung (F4,F6,F7,F16,F17) → 162,5 Personenstunden
- Z2: Ausführung von Algorithmen (F5,F9,F1,F2,F13,F15) → 101,25 Personenstuden
- Z3: Erweiterte Diagramme (F3,F8,F11,F12,F14) → 101,25 Personenstunden
- Z4: Quality of life features (F10,F18,F19) → 40,5 Personenstunden

# Glossar

**.csv** Dateiformat zum Verwalten von Text- und Zahlenwerten in Form von Tabellen. Wird im Programm für den Import und Export der Bounds eines Netzes benutzt.

**.onnx** Open Neural Network Exchange, quelloffenes Dateiformat zum Speichern von Neuronalen Netzen. Kann zum Auswerten importiert werden..

**.py** Dateiformat eines Pythonskriptes. Kann genutzt werden, um eigene Algorithmen in die Software einzubinden.

**.svg** Dateiformat für Vektorgrafiken. Bilder werden nicht als als einzelne Pixel gespeichert, sondern als mathematisch definierte Objekte wie Linien, Kreise und Kurven.

**.vnnlib** Beinhaltet die Input- und Output Bounds eines Netzwerks, nicht aber das Netzwerk selbst.

**Bibliothek** Ansammlung an Softwarecode anderer Entwickler, die diesen als Paket zur Einbindung in fremde Projekte zur Verfügung stellen..

**Binding** Verbindung zwischen zwei Softwarekomponenten, bspw. zwischen dem Programm und einer in einer anderen Sprache geschriebene Bibliothek..

**Caching** Die Speicherung zuvor geladener oder berechneter Daten im Zwischenspeicher, ermöglicht einen schnellen erneuten Zugriff auf diese.

**Code Coverage Tool** Tool zur Erfassung davon, wie viel des Codes durch Tests abgedeckt ist.

**deterministischer Algorithmus** Algorithmus, der bei gleicher Eingabe immer zum gleichen Ergebnis kommt.

**Diagramm** Stellt den Zusammenhang eines Neuronenpaares mit einem Algorithmus dar. Kann entweder als einzelner Tab oder als Teil eines Vergleiches angezeigt werden.

**draggen** Bewegen der Maus während ein Button gedrückt gehalten wird. Geste, um UI-Elemente zu verschieben oder im dreidimensionalen Raum zu drehen..

**Dropdown-Menü** Ein UI-Element zum Auswählen eines Entrags aus einer vorgegebenen Liste an Optionen, wird zur Auswahl von Neuronalen Netzen und Algorithmen benutzt.

**einsatzbereites Netz** Ein Netz, bei dem Eingabebounds schon geladen/eingestellt wurden.

**großes Netz** Ein Netz mit mindestens 100000 Knoten.

**GUI** Graphical User Interface - Die Benutzeroberfläche des Programms.

**Layer** Einzelne Schicht eines neuronalen Netzes. Es gibt einen Input- und einen Outputlayer für die Ein- und Ausgabe des Netzes.

**Log-Datei** Datei zur Speicherung von Systemnachrichten.

**MemoryError** Ein speicherbedingter Fehler.

**Neuron** Ein einzelner Knoten eines Netzwerkes. Wird über gewichtete Kanten mit anderen Neuronen verbunden.

**Neuronales Netz** Netz aus künstlichen Neuronen, an biologischen Gehirnen inspiriert. Besteht aus Nodes in Layern und gewichteten Verbindungen.

**Open-Source-Lizenz** Eine Lizenz für quelloffene Software.

**pan** Bewegen eines UI-Elements durch Draggen der Maus in zwei Dimensionen. Wird für die Navigation von Diagrammen und Neuronalen Netzen benutzt.

**PEP 8** Ein Style Guide für Python Code.

**Polytop** Eine Verallgemeinerung von konvexen Vielecken in beliebiger Dimension.

**Session** Engl. für Sitzung, beschreibt die Zeitperiode ab dem Öffnen des Programms durch einen Nutzer bis zum vollständigen Schließen..

**Standartnetz** Ein Netz mit 2000 Neuronen, die auf 4 Layer verteilt sind.

**System Usability Scale** Tool zur Erfassung der subjektiv empfundenen Usability. , 19

**Tab** Ein "karteikartenartiger" Reiter in einer Auflistung, wird zum Wechseln der Ansicht zwischen verschiedenen Neuronalen Netzen, Diagrammen und Vergleichen verwendet.

**Unit Test** Test zur Überprüfung eines klar abgegrenzten Teils des Programms.

**Vergleich** Ein Vergleich zeigt Diagramme mehrerer Neuronenpaare.

**Zonotop** Ein Zonotop ist die Minkowski-Summe einer endlichen Anzahl an Vektoren. Die enstandene Fläche ist konvex. Es wird im Diagramm dargestellt und aus neuronalen Netzen generiert.

**zoom** Ändern der Anzeigegröße eines UI-Elements. Wird für die Navigation von Diagrammen und Neuronalen Netzen benutzt.